JIRA SOFTWARE

The Practical Handbook

Beginner → Intermediate → Advanced

A structured guide to planning, tracking, workflows, JQL, automation, reporting, administration, real-world usage, and interview preparation.

Designed for	Scope	Edition
New Jira users, developers, QA, scrum masters, project leads, admins, and interview candidates	Primarily Jira Cloud / Jira Software concepts; notes flag where configuration or product version may differ	October 2026

How to use this handbook: read Parts 1–4 to become productive, use Parts 5–8 as a reference at work, then practise the scenarios and interview questions. Jira labels and navigation can change between Cloud, Data Center, plans, and company configuration.

Prepared as a practical learning document. Product behavior should be verified against your organisation’s Jira configuration and current Atlassian documentation.

Contents

1. Jira in plain language

2. Core objects and terminology

3. Choose the right project and board

4. Create and manage work items

5. Agile with Scrum and Kanban

6. Workflows, statuses, transitions, and resolutions

7. Search: basic search, filters, and JQL

8. Advanced Jira features and integrations

9. Automation: triggers, conditions, actions, and smart values

10. Reports, dashboards, and delivery metrics

11. Permissions, schemes, and administration

12. Practical end-to-end workflow for a software team

13. Best practices, anti-patterns, and troubleshooting

14. Interview questions with model answers

15. Practice exercises and quick-reference checklist

16. Glossary and official learning resources

1. Jira in plain language

Jira is a work-tracking platform used to plan, assign, prioritise, execute, and report on work. Software teams commonly use Jira Software for Agile delivery, bug tracking, release coordination, and linking work to code and deployments. Jira is configurable: the same label or workflow may behave differently across organisations.

Why use Jira?

- **Visibility:** see what is planned, in progress, blocked, and done.
- **Ownership:** identify who is responsible and who needs to act next.
- **Traceability:** connect requirements, bugs, decisions, code changes, testing, and releases.
- **Predictability:** use backlogs, estimates, sprint history, and flow metrics to improve planning.
- **Repeatability:** standardise common processes with workflows, permissions, and automation.
- **Communication:** keep status, decisions, evidence, and next steps near the work.

When should Jira be used?

Use Jira when…	Avoid using Jira as…
Work needs an owner, status, priority, history, or cross-team visibility.	A substitute for every conversation, design tool, source-control system, or knowledge base.
Teams need a shared backlog, sprint, Kanban flow, defect queue, or release plan.	A dumping ground for vague tasks that nobody reviews.
Auditability and traceability matter, such as engineering or regulated delivery.	A proxy for productivity based only on ticket counts.

Cloud versus Data Center

Jira Cloud is Atlassian-hosted and updated as a service. Jira Data Center is self-managed enterprise deployment. Menus, administration, integrations, feature availability, terminology, and release cadence can differ. This handbook focuses on common Jira Cloud workflows and concepts; confirm Data Center-specific steps in the documentation for your installed version.

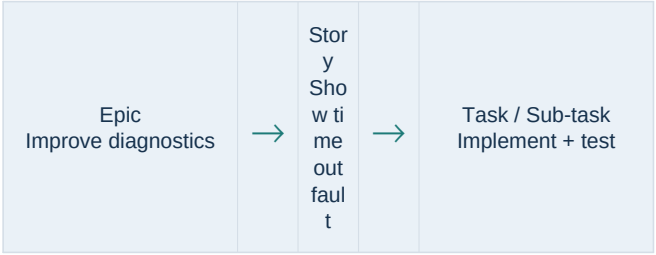
Core idea: Jira does not create a good process by itself. It makes the process visible and repeatable. A simple, understood workflow usually beats a complex workflow nobody follows.

2. Core objects and terminology

Concept	Meaning	Example / use
Site / organisation	The Jira environment and its administrative boundary.	company.atlassian.net
Project / space	A container for work, settings, and team processes; Atlassian UI terminology may vary by product and rollout.	Firmware Platform
Work item (formerly issue)	A trackable unit of work.	JRA-123: fix sensor timeout
Work item key	Unique project-prefixed identifier.	FW-248
Issue type / work type	Classifies the work.	Epic, Story, Task, Bug, Sub-task
Summary	Short title describing the work.	Add CAN bus timeout diagnostic
Description	Context, requirements, evidence, and expected outcome.	Reproduction steps and acceptance criteria
Assignee	Person currently responsible for moving the work forward.	Developer or QA engineer
Reporter / creator	Person who reported or created the item; distinct from assignee.	QA reports a defect
Priority	Relative urgency/importance as configured by the team.	Highest, High, Medium, Low
Status	Current workflow state.	To Do, In Progress, In Review, Done
Resolution	Outcome field indicating how work was resolved, where used.	Fixed, Won't Do, Duplicate
Component	Logical subsystem or area of a project.	Bootloader, UI, CAN driver
Label	Flexible tag for grouping/searching.	regression, customer-impact
Fix Version / release	Release/version a work item is intended to ship in.	v2.4.0
Sprint	A time-boxed iteration used by Scrum teams.	Sprint 18
Board	A view of work based on a filter and board configuration.	Scrum board / Kanban board
Backlog	Ranked work awaiting selection or execution.	Candidate stories
Epic	A larger initiative grouping related work.	Improve diagnostics
Link / dependency	Relationship between work items.	blocks / is blocked by
Watcher / comment / history	Collaboration and audit trail.	Follow changes and explain decisions

A useful hierarchy (illustrative)

From initiative to executable work



A hierarchy helps connect strategy to implementation. Exact parent-child relationships and supported levels depend on Jira configuration and product version. Use links for dependencies that are not true parent-child relationships.

What makes a good work item?

- A clear, action-oriented summary.
- Why the work matters and enough context to act without repeated clarification.
- Acceptance criteria or a measurable definition of success.
- Priority, owner, component, and target release when relevant.
- Dependencies, risks, logs, screenshots, test evidence, and links where useful.
- A status that reflects reality—not an optimistic guess.

3. Choose the right project and board

Common project templates

Template / approach	Use it when	Trade-off
Scrum	The team plans a goal and a selected set of work for fixed-length sprints.	Needs backlog refinement, sprint planning, and regular review.
Kanban	Work arrives continuously; flow and work-in-progress limits matter more than sprint commitments.	Needs explicit policies and discipline to prevent work piling up.
Bug tracking	The main need is to record, triage, assign, and resolve defects.	May lack the planning structure needed for a broader product roadmap.
Team-managed	A team wants simpler local configuration and autonomy.	May be less suited to shared, centrally governed configuration.
Company-managed	Admins need standardised schemes/workflows across teams or projects.	More powerful but configuration changes can affect multiple teams.

Project/space terminology and template choices evolve. Check the current UI and your organisation’s conventions before creating a new project.

Board, project, and filter are different

- **Project/space:** stores work and configuration.
- **Board:** visualises work matching a filter; board columns map statuses into a workflow view.
- **Filter:** a saved search that defines which work items are returned.
- **Backlog:** planning view for prioritising work; available capabilities depend on board type.

A board is not necessarily identical to a project. One board can show work from several projects, and a project can be viewed by multiple boards, if permissions and filters allow it.

Setup checklist

- Confirm the team’s purpose and working method before choosing a template.
- Agree on a small set of work types and fields.
- Define statuses and what each status means.
- Set board columns to reflect meaningful stages, not every tiny activity.
- Agree on ownership, triage, priority, and definition of done.
- Test permissions and filters with normal team accounts.

4. Create and manage work items

A practical bug report template

```
Summary: [Subsystem] - [observable failure]
Environment: [HW/SW version, build, configuration]
Preconditions: [setup needed]
Steps to reproduce:
1. ...
2. ...
Expected result: ...
Actual result: ...
Frequency: [always / intermittent / once]
Impact / severity: ...
Evidence: [logs, trace, photo, test report]
Acceptance criteria: ...
```

User story template

```
As a [user/persona],
I want [capability],
so that [benefit].

Acceptance criteria:
- Given [context], when [action], then [observable result].
```

Not every task should be forced into user-story language. A technical task, investigation, spike, or maintenance item can use a direct description with explicit outcome and boundaries.

Priority versus severity

Severity describes impact or technical seriousness; **priority** describes the order in which the team should act. A critical production failure may be both high severity and high priority, but a severe issue with a rare workaround might be prioritised differently. Agree definitions locally.

Sub-tasks, components, labels, and links

- Use sub-tasks when a parent work item can be split into separately owned pieces that still belong to the same outcome.
- Use components for stable product/system areas; use labels for flexible cross-cutting tags.
- Use issue links for relationships such as blocks, is blocked by, relates to, or duplicates.
- Avoid creating a sub-task for every tiny action; too much granularity creates maintenance overhead.

Definition of Ready and Definition of Done

Definition of Ready (example)	Definition of Done (example)
Problem and expected outcome are understood.	Implementation reviewed and merged.
Acceptance criteria and test approach are clear.	Automated/manual tests passed as required.
Dependencies and risks are identified.	Documentation and diagnostics updated where needed.
Item is small enough to plan realistically.	Required approvals and release conditions met.

These are team agreements, not magic Jira defaults. Store them in project guidance, templates, or linked documentation.

5. Agile with Scrum and Kanban

Scrum: typical cycle

A sprint cycle



The sprint is time-boxed. The team forecasts a realistic amount of work, works toward a sprint goal, reviews the result, and improves its way of working.

How to use Scrum in Jira

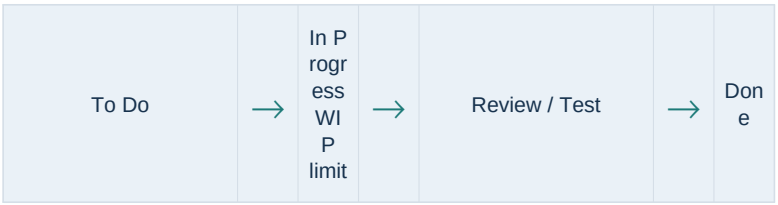
- Create and rank backlog items; clarify acceptance criteria and dependencies.
- Estimate items using the team's chosen method (story points or time-based estimates).
- Create a sprint, agree on a goal, and select work based on capacity and priority.
- Start the sprint; move cards as work actually changes state.
- Use the board and burndown/burnup or sprint reports to inspect progress—not to punish individuals.
- At sprint end, complete the sprint and decide where unfinished work should go.
- Review what was delivered and capture one or two concrete improvement actions.

Story points versus time estimates

Story points are a team-relative estimate of effort/complexity/uncertainty; they are not hours and should not be converted into a universal hours-per-point formula. Time estimates are useful when the work needs explicit duration/capacity planning. Estimation is for forecasting, not a guarantee.

Kanban: continuous flow

A simple Kanban flow



Kanban focuses on flow, visualising work, limiting work in progress (WIP), and improving bottlenecks. A WIP limit is a policy: if the limit is reached, the team helps finish existing work instead of starting more.

When to choose which?

Question	Scrum tends to fit	Kanban tends to fit
Work intake	Can plan a sprint goal and forecast.	Arrives unpredictably or continuously.
Planning rhythm	Regular sprint planning/review.	Replenish work when capacity opens.
Primary signal	Sprint goal, forecast, completed increment.	Cycle time, throughput, aging WIP, blocked work.
Common pitfall	Overcommitting or changing scope constantly.	No WIP limits or unclear pull policies.

Velocity and flow metrics

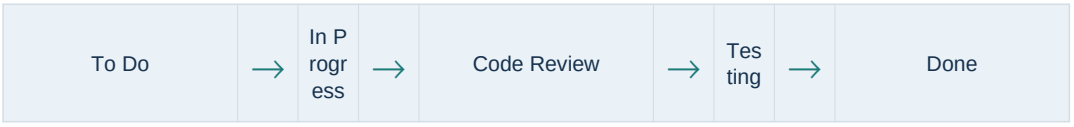
Velocity is the amount of estimated work completed per sprint, using a consistent team convention. It is useful for forecasting for that team, not comparing teams. Kanban teams often use throughput (items completed per period), cycle time (start to finish), work-item age, and cumulative flow to spot delays.

Metrics should trigger questions, not blame. If a chart looks bad, investigate scope changes, blockers, dependencies, item size, and system constraints before judging performance.

6. Workflows, statuses, transitions, and resolutions

A workflow defines the allowed path of a work item. A status tells you where it is; a transition is the action that moves it to another status. Conditions, validators, and post-functions or equivalent rules can govern what happens during a transition, depending on Jira deployment and configuration.

Example engineering workflow



Workflow concepts

Term	Purpose	Example
Status	Current state.	In Progress
Transition	Allowed move between states.	Submit for review
Condition	Who may execute a transition.	Only assignee or reviewer role
Validator	What must be true before transition succeeds.	Resolution notes required
Post-function / rule	Action performed during/after transition.	Set resolution or assign owner
Resolution	Outcome of completed work, where configured.	Fixed / Won't Fix / Duplicate

Why status and resolution are not the same

A status such as Done describes workflow position. Resolution describes the outcome. In workflows that use a resolution field, an item can look Done while resolution is empty, or resolution can remain set after reopening if the workflow is poorly configured. This can distort reports and JQL such as unresolved-work searches.

Workflow design guidance

- Use statuses that represent meaningful states or queues.
- Make transition names describe an action: Start work, Request review, Reopen.

- Define who can move work and what evidence is required.
- Ensure completion sets the correct resolution; reopening clears it when appropriate.
- Avoid adding statuses merely to record every meeting or handoff.
- Test the workflow using normal user permissions and representative work types.

Example policy

Move to Done only when acceptance criteria are met, required review is complete, test evidence is attached or linked, and the resolution is set correctly. If a release is required, use a separate release readiness checklist rather than assuming Done means deployed.

7. Search: basic search, filters, and JQL

Jira provides quick/basic search and advanced search. Jira Query Language (JQL) is a structured query language for finding work items; it is not SQL and generally does not create, update, or delete records.

JQL building blocks

Element	Meaning	Example
Field	What to search.	project, status, assignee, priority, labels
Operator	How to compare.	=, !=, IN, NOT IN, ~, IS, IS NOT, >, <
Value	Target value.	'In Progress', currentUser()
AND / OR / NOT	Combine or negate conditions.	project = FW AND priority = High
ORDER BY	Sort results.	ORDER BY priority DESC, updated DESC

Useful starter queries

```
project = FW ORDER BY created DESC

assignee = currentUser() AND statusCategory != Done ORDER BY priority DESC

project = FW AND status = "In Progress" ORDER BY updated ASC

project = FW AND issuetype = Bug AND resolution IS EMPTY

project = FW AND priority in (Highest, High) AND statusCategory != Done

project = FW AND labels = regression ORDER BY created DESC

project = FW AND updated >= -7d ORDER BY updated DESC

project = FW AND sprint in openSprints() ORDER BY Rank ASC

project = FW AND statusCategory = Done AND resolved >= startOfMonth()

project = FW AND assignee IS EMPTY AND statusCategory != Done
```

Replace FW and field values with your project's actual keys and configured values. Some functions and fields vary by deployment, permissions, and configuration.

Important JQL details

- Use quotes around multiword values, for example status = "In Review".
- AND usually narrows results; OR broadens them. Use parentheses when combining logic.
- ORDER BY Rank ASC is commonly used to display backlog order, but board ranking behavior depends on board configuration.

- resolution IS EMPTY is not always equivalent to statusCategory != Done; understand your workflow's resolution behavior.
- A saved filter can power a dashboard, subscription, or board. Sharing a filter does not automatically grant access to every work item.

Example: find stale work

```
project = FW
AND statusCategory != Done
AND updated <= -7d
ORDER BY updated ASC
```

Use this query for a review queue, then decide whether the work is blocked, waiting for input, no longer needed, or simply missing updates. Staleness is a prompt to investigate, not proof that someone is inactive.

Troubleshooting a query

- Confirm the project key and exact status/type values.
- Check permissions: JQL cannot return work you are not allowed to see.
- Add clauses one at a time to find the condition excluding results.
- Check whether fields are searchable and whether the correct operator is supported.
- Save a clear filter name and share it with the intended audience.

8. Advanced Jira features and integrations

Versions and releases

A version/release groups work intended for a release. Fix Version typically expresses the release a bug or change is expected to be delivered in; Affects Version records the release(s) where a defect is observed, where those fields are configured. Release readiness should include unresolved work, test status, rollout plan, known issues, and stakeholder approval.

Components, labels, and custom fields

- Components suit relatively stable subsystems and can support ownership/reporting.
- Labels are flexible tags; uncontrolled spelling variants reduce search quality.
- Custom fields should answer a real reporting, routing, compliance, or workflow need. Each new field increases user effort and administration.

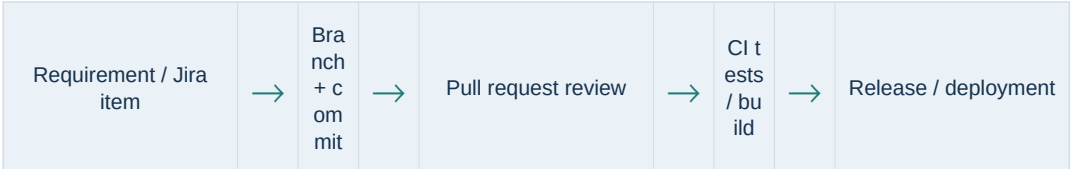
Issue links and dependencies

Use explicit links to express relationships across teams or projects. A dependency should have an owner, impact, and expected date where relevant. Avoid relying on comments alone for critical blockers.

Development integrations

Depending on connected tools and permissions, Jira work items can link to branches, commits, pull requests, builds, and deployments. A common pattern is to include the Jira key in a branch name or commit/PR reference, for example FW-248-add-timeout-diagnostic. Integration behavior depends on the source-control and CI/CD products, app configuration, and access rights.

Traceability chain



Forms, templates, components, and reusable processes

Use templates or standard descriptions to collect the same information for recurring work. For example, a defect template can request build number, hardware revision, reproduction steps, logs, expected result, and actual result. Keep required fields minimal enough that users complete them correctly.

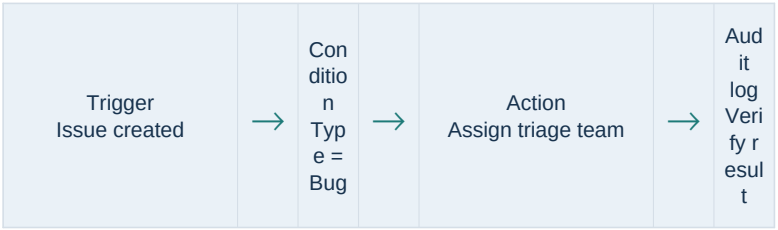
When should functionality be used?

Functionality	Use it for	Do not overuse it for
Epic	A coherent initiative spanning several smaller work items.	A catch-all bucket for unrelated tasks.
Sub-task	Separately trackable work under a parent outcome.	Every tiny checklist action.
Component	Stable technical area with ownership/reporting value.	Temporary campaign tags.
Label	Flexible cross-cutting categorisation.	A substitute for controlled fields when reporting must be reliable.
Fix Version	Target release tracking.	A speculative version assigned to every idea.
Issue link	Dependencies or relationships across work items.	Relationships already represented by a proper parent-child hierarchy.
Custom field	A real business/process requirement.	Collecting information no one uses.

9. Automation: triggers, conditions, actions, and smart values

Automation rules reduce repetitive work and improve consistency. A typical rule has a trigger, optional conditions, and one or more actions. In Jira Cloud, rule availability, limits, UI labels, and smart-value behavior can vary by plan and product version.

Automation rule anatomy



Useful examples

Use case	Trigger	Condition / action idea
Auto-assign component owner	Work item created or component changed	If component matches, assign its owner.
Notify when blocked	Status changes or label added	If blocked, notify the team channel or watchers.
Require triage consistency	Work item created	Set a default priority or add a triage label when appropriate.
Close parent when all children are done	Child work item transitioned	Check related children, then update parent if policy allows.
Stale-item reminder	Scheduled trigger	Find unresolved work not updated for N days; notify owner or queue.
Release checklist	Fix Version or status changes	Create/notify checklist owner; avoid declaring release safe automatically.

Smart values

Smart values are expressions used in automation to insert dynamic work-item or rule data into fields and messages. Exact available values depend on the rule context. Test expressions in the audit log and official documentation rather than guessing.

```
Example message text:
Work item: {{issue.key}} - {{issue.summary}}
Assignee: {{issue.assignee.displayName}}
Status: {{issue.status.name}}
```

Automation safety checklist

- Start with a clear problem statement and one rule.
- Choose the narrowest trigger and add explicit conditions.
- Check for loops: an action that edits an item may trigger the same rule again.
- Test with sample work items before enabling broadly.
- Inspect audit logs and rule actor permissions.
- Document the owner and purpose; remove rules that are redundant or unmaintained.
- Be careful with bulk changes, notifications, assignments, and transitions.

Automation should enforce an agreed process, not hide a broken one. If people disagree about what 'Done' means, automating Done transitions will scale the disagreement.

10. Reports, dashboards, and delivery metrics

Common reports and what they answer

Report / view	Question it helps answer	Caution
Burndown	How much estimated work remains in a sprint?	Scope changes can distort the line.
Burnup	How much work is complete versus total scope?	A rising scope line may explain apparent slippage.
Velocity	How much estimated work has the team completed per sprint?	Use for the same team and estimation convention.
Cumulative flow diagram	Where is work accumulating across workflow states?	Column mapping and status discipline matter.
Control chart	How long does work take through the system?	Compare similar work types and understand outliers.
Created vs resolved	Are new items arriving faster than they are resolved?	Not a complete measure of quality or value.
Dashboard	Can stakeholders see the signals relevant to their role?	Too many gadgets create noise.

Dashboard design

- Start with the audience: engineer, team lead, product owner, manager, or support.
- Use a few meaningful gadgets and saved filters, such as My Open Work, Blocked Items, High-Priority Bugs, and Recently Resolved.
- Make ownership and next action visible.
- Use filters that respect permissions and are maintained by an accountable owner.
- Review dashboard usefulness periodically; remove stale charts and duplicated signals.

Good metrics versus vanity metrics

Useful signal	Why it helps
Cycle time / age of work	Shows delay and items that may need intervention.
Escaped defects / reopen rate	Provides a quality signal when definitions are consistent.
Blocked time and dependency age	Reveals external constraints and handoff delays.
Sprint goal outcome	Checks whether the sprint delivered its intended value.
Throughput plus work type/context	Helps understand delivery flow without treating all items as equal.

Avoid ranking individuals by ticket count, story points, or time logged. Such measures are easy to game and do not capture complexity, review, collaboration, quality, or customer value.

11. Permissions, schemes, and administration

Jira access commonly involves several layers: global/site permissions, project/space permissions, and issue/work-item-level security. The exact controls vary by deployment and configuration.

Layer	Controls	Question
Global / site	Who can administer or access site-wide capabilities?	Who can manage users, apps, or global settings?

Layer	Controls	Question
Project / space	Who can browse, create, edit, assign, transition, comment, or administer project settings?	Can this team member perform the required action here?
Work-item security	Which users/roles can view a particular item?	Is this sensitive item hidden from otherwise authorised project users?
Workflow	Who can execute transitions, and what must be true?	Can a user move this item to this state?

Schemes and configuration

In company-managed configurations, schemes can associate shared settings such as permissions, notifications, issue types, fields, and workflows with projects. Team-managed projects typically expose more local configuration. Exact terminology and sharing behavior depend on Jira version and project type.

Least privilege

- Give users only the access required for their role.
- Prefer groups and project roles over one-off user grants when practical.
- Review admin access and external collaborators periodically.
- Test access with a normal account—not only an administrator account.
- Be cautious with issue security, since it can hide work from reports and teams.
- Document who owns configuration and how changes are reviewed.

Admin change checklist

Before changing a workflow, field, permission scheme, board filter, or automation rule, identify every project/team affected, define the expected behavior, test in a safe environment where possible, communicate the change, and prepare a rollback plan.

12. Practical end-to-end workflow for a software team

Scenario: a firmware team needs to fix intermittent CAN communication timeouts and ship the change in release 2.4. The same process can be adapted to web, mobile, platform, or embedded products.

Step 1 — Capture the defect

- Create a Bug with the affected build, hardware revision, reproduction rate, expected/actual behavior, logs, and impact.
- Link the customer report or test run if appropriate; avoid placing secrets or sensitive data in unrestricted comments.

Step 2 — Triage

- Confirm severity and priority; assign the triage owner and component (e.g., CAN driver).
- Check for duplicates and related work; set Affects Version if known and configured.

Step 3 — Plan the fix

- Write acceptance criteria: timeout is detected, diagnostic is recorded, recovery behavior is defined, regression tests pass.
- Create a story/technical task or sub-tasks for implementation, test updates, and documentation if separately owned.
- Add dependencies and estimate using the team’s chosen method.

Step 4 — Implement and review

- Move the item to In Progress when work starts, not when it is merely assigned.
- Reference the Jira key in branch/PR conventions; link commits and build results when integrations support it.

- Record decisions and blockers in the item; request review and test evidence.

Step 5 — Verify and complete

- Test the acceptance criteria on relevant hardware/configurations and record evidence.
- Resolve the defect according to the team's policy; set the correct resolution if used.
- Assign Fix Version 2.4 where appropriate and confirm release readiness separately.

Example defect lifecycle

Reported	→	Triaged	→	In Progress	→	Review	→	Testing	→	Done / Resolved

Step 6 — Learn

Review cycle time, escaped defects, reopened items, and the cause of the timeout. Capture a small improvement action—for example, add a bus-load stress test or improve diagnostic logging. Jira records the work; the engineering team must still analyse the technical cause.

Example acceptance criteria

```
Given the controller is operating under supported bus-load conditions,
when no valid message arrives within the configured timeout,
then the firmware records the diagnostic with a timestamp,
exposes the agreed fault state, and follows the documented recovery policy.

And the regression test passes on the supported hardware variants.
```

Keep acceptance criteria observable and testable. Avoid vague criteria such as 'works properly' or 'improves reliability' without a measurable condition.

13. Best practices, anti-patterns, and troubleshooting

Best practices

- Use a clear summary and a single accountable assignee for the next action.
- Keep status current; explain blockers and name the dependency owner.
- Write acceptance criteria before implementation becomes expensive.
- Use a consistent resolution policy and periodically check Done items with empty resolution.
- Use components and controlled fields for reliable reporting; use labels deliberately.
- Keep workflows and required fields as simple as the team can sustain.
- Use automation for stable, repetitive rules with visible logs and an owner.
- Use dashboards to answer specific questions, not to display every possible metric.
- Review stale filters, dashboards, rules, users, and configuration regularly.

Anti-patterns

Anti-pattern	Better approach
Everything is Highest priority	Agree definitions and triage using impact, urgency, and risk.
A ticket stays In Progress for weeks	Split work if useful; expose blockers and next action; review aging work.
Too many statuses	Represent only meaningful states, queues, or approvals.
Story points used to rank individuals	Use estimates for team forecasting and discuss delivery context.
One huge ticket covers a whole release	Break down by testable outcomes and ownership.
Automation silently changes many items	Narrow scope, test, inspect audit logs, and document rollback.
Done means code was written	Define completion to include review, tests, evidence, and required documentation.
A board shows no work unexpectedly	Check board filter, project scope, status-column mapping, and permissions.

Troubleshooting quick guide

Symptom	Likely checks
Cannot see a work item	Project access, issue security, filter scope, correct project key.
Cannot transition status	Transition exists, permission/condition, required fields/validators.
Work item appears Done but reports unresolved	Resolution field and workflow post-functions/rules.
Board card missing	Board filter, saved filter permissions, sub-filter, status mapping, quick filters.
JQL returns zero results	Project key, exact values, quotes, permissions, clause-by-clause testing.
Automation did not run	Rule enabled, trigger event, conditions, actor permissions, scope, audit log, limits.
Sprint report looks wrong	Board filter, estimation statistic, issue movement, scope changes, status mapping.
Email/notifications absent	Notification scheme, user preferences, watchers, permissions, event/rule setup.

A disciplined debugging method

Reproduce the issue → capture the exact key/action/time → check permissions and configuration → isolate one condition at a time → inspect history/audit logs → test the fix on a small example → document the cause and prevention.

14. Interview questions with model answers

Beginner questions

1. What is Jira?

A configurable work-tracking platform used to manage tasks, defects, planning, workflows, collaboration, and reporting.

2. What is a work item/issue?

A record representing a unit of work, with fields such as summary, description, status, assignee, priority, history, and links.

3. What is the difference between a project and a board?

A project/space contains work and configuration; a board is a view of work selected by a filter and mapped into columns.

4. What is a backlog?

A prioritised list of work that may be selected for execution. Scrum backlogs commonly support sprint planning.

5. What is an epic?

A larger initiative that groups related work items. Its exact hierarchy depends on configuration.

6. What is the difference between assignee and reporter?

The assignee is responsible for the next action; the reporter is the person who reported or created the item.

7. What is the difference between priority and severity?

Severity describes impact; priority describes order of action. Teams should define both consistently.

8. What is a workflow?

The statuses and allowed transitions through which a work item moves.

9. What is a sprint?

A fixed-length iteration in Scrum during which a team works toward a sprint goal.

10. What is Kanban?

A flow-based approach that visualises work, limits WIP, and improves throughput and cycle time.

Intermediate questions

11. What is JQL?

Jira Query Language is used to search for work items with structured conditions, operators, functions, and sorting.

12. What is the difference between status and resolution?

Status is workflow state; resolution is the outcome field used by workflows/reports where configured. They should be kept consistent.

13. What is the difference between a component and a label?

Components are structured project areas and may have owners; labels are flexible tags.

14. What is a Fix Version?

The release/version a work item is expected to be delivered in, where that field is used.

15. What are story points?

A relative team estimate of effort, complexity, and uncertainty; they are not universally equivalent to hours.

16. What is velocity?

The amount of estimated work completed per sprint using a consistent team convention; it supports local forecasting, not cross-team ranking.

17. What is a saved filter?

A stored search that can be reused in views, dashboards, subscriptions, or board configuration.

18. What is a sub-task?

A separately trackable piece of work under a parent item; use it when separate ownership or tracking is useful.

19. What are issue links?

Relationships such as blocks, is blocked by, relates to, or duplicates.

20. How would you find unassigned open bugs in a project?

Example JQL: project = FW AND issuetype = Bug AND assignee IS EMPTY AND statusCategory != Done. Replace FW with the actual project key.

Advanced / administrator questions

21. Explain conditions, validators, and post-functions.

Conditions control who can execute a transition; validators check requirements before it succeeds; post-functions or equivalent rules perform actions as part of/after the transition. Exact implementation varies by deployment.

22. How do you troubleshoot a missing board card?

Check the board filter and its permissions, project scope, sub-filter/quick filters, status-to-column mapping, and whether the user can browse the item.

23. Why can Done items appear in unresolved searches?

The workflow may not set the resolution field when completing the item. Review workflow transitions and resolution-setting behavior.

24. How do you design a safe automation rule?

Define the use case, use a narrow trigger and explicit conditions, avoid loops, test with sample items, check actor permissions, review audit logs, and document ownership.

25. What are the layers of Jira permissions?

Commonly global/site, project/space, and work-item security, with workflows also controlling transitions. Exact details depend on product/configuration.

26. What is the difference between team-managed and company-managed?

Team-managed projects generally favour local team autonomy; company-managed projects support more centrally shared configuration. Check current product behavior for exact capabilities.

27. How would you reduce a bloated Jira workflow?

Map real states and handoffs, remove redundant statuses/transitions, clarify ownership and completion criteria, test reporting effects, and roll out with communication.

28. How do you improve sprint predictability?

Refine backlog items, clarify dependencies, use historical team data, account for capacity, protect the sprint goal, and inspect scope changes and carryover.

29. How do you measure Kanban flow?

Use cycle time, throughput, work-item age, WIP, and cumulative flow; interpret them alongside item type, blocked time, and quality.

30. How do you make Jira useful for embedded/firmware engineering?

Capture hardware/software versions, reproduction steps, logs and traces, subsystem/component, acceptance criteria, test evidence, and links to commits/builds/releases. Track hardware-dependent validation explicitly.

Scenario-based interview prompts

A critical bug is marked Done but the release is not shipped. Is this wrong?

Not necessarily. Done may mean implementation and validation are complete; release/deployment can be a separate state or process. Make the distinction explicit in team policy.

A developer has many tickets but the sprint goal failed. What do you inspect?

Goal clarity, dependencies, scope changes, ticket sizes, review/test bottlenecks, and whether completed work delivered the intended outcome—not just ticket count.

An automation rule repeatedly re-triggers itself. What do you do?

Inspect the audit log, identify which action re-triggers the rule, add conditions or guard fields, use appropriate loop prevention, and test on a small scope.

Two teams want different workflows in one project. What would you consider?

Clarify reporting and ownership needs, evaluate separate projects/boards or compatible workflow schemes, and avoid forcing unrelated processes into one workflow.

15. Practice exercises and quick-reference checklist

Practice lab

Exercise 1 — Create a project

Create a test project using a suitable template. Identify project settings, board, backlog, and issue types.

Exercise 2 — Create a well-formed bug

Use the bug template in Chapter 4. Add acceptance criteria, component, priority, and evidence.

Exercise 3 — Build a board

Create a saved filter for open work and verify which statuses map to each column.

Exercise 4 — Practise JQL

Find unassigned bugs, stale open work, high-priority items, and items updated in the last seven days.

Exercise 5 — Workflow reasoning

Draw your team’s current statuses. Mark who owns each transition and what evidence is needed to finish.

Exercise 6 — Automation

Create a safe rule in a test scope to notify a team when a work item becomes blocked. Inspect the audit log.

Exercise 7 — Dashboard

Create a small dashboard with My Open Work, High-Priority Bugs, and Stale Work filters.

Exercise 8 — End-to-end traceability

Link a sample work item to a branch/PR/build or add mock references in the description; document release readiness.

Daily team-member checklist

- Is my assigned work item current and correctly prioritised?
- Does the status reflect actual work?
- If blocked, is the blocker documented with an owner and next action?
- Are acceptance criteria and test evidence clear?
- Are relevant dependencies and code/release links included?
- Does the item meet the team’s Definition of Done before completion?

Team lead / project owner checklist

- Is the backlog ranked and sufficiently refined?
- Are high-risk dependencies and aging work visible?
- Do board columns and workflow statuses match real work?
- Are sprint goals realistic and reviewed?
- Are dashboards based on maintained filters and useful questions?

• Are automation rules, permissions, and custom fields still needed?

JQL quick reference

Need	Query pattern
My unresolved work	assignee = currentUser() AND statusCategory != Done
Unassigned items	assignee IS EMPTY AND statusCategory != Done
High priority	priority in (Highest, High) AND statusCategory != Done
Recently updated	updated >= -7d ORDER BY updated DESC
Stale unresolved work	statusCategory != Done AND updated <= -7d ORDER BY updated ASC
Unresolved bugs	issuetype = Bug AND resolution IS EMPTY
Current sprint	sprint in openSprints()
Resolved this month	statusCategory = Done AND resolved >= startOfMonth()

Treat these as patterns, not universal copy/paste commands. Confirm exact field names, status categories, issue types, and functions in your Jira instance.

16. Glossary and official learning resources

Term	Plain-language definition
Agile	An iterative approach that delivers value, learns from feedback, and adapts.
Acceptance criteria	Observable conditions that must be met for work to be accepted.
Backlog refinement	Ongoing clarification, prioritisation, and sizing of upcoming work.
Blocked	Work cannot progress until a dependency or impediment is resolved.
Board filter	The saved search defining which work items appear on a board.
Cycle time	Elapsed time from a defined start point to a defined completion point.
Definition of Done	Team agreement describing the quality/completion bar.
Epic	A larger initiative grouping related work.
JQL	Jira Query Language for searching work items.
Kanban	A flow-based approach using visualisation and WIP management.
Resolution	Outcome recorded for a work item in workflows that use the field.
Scrum	An Agile framework using sprints, a goal, inspection, and adaptation.
Sprint goal	The outcome the team aims to achieve during a sprint.
Throughput	Number of work items completed in a period.
WIP	Work in progress; the amount of work currently underway.
Workflow transition	An allowed action that moves a work item from one status to another.

Official resources

• **Getting started with Jira**
https://www.atlassian.com/software/jira/guides/getting-started/basics

• **Atlassian Jira Agile tutorials**
https://www.atlassian.com/agile/tutorials/

- **Create and plan work with Scrum and Kanban (Cloud)**
https://support.atlassian.com/jira-software-cloud/docs/create-and-plan-work-with-scrum-and-kanban/

- **Estimate a work item (Cloud)**
https://support.atlassian.com/jira-software-cloud/docs/estimate-an-issue/

- **Coordinate and monitor work with sprints (Cloud)**
https://support.atlassian.com/jira-software-cloud/docs/coordinate-and-monitor-work-with-sprints/

- **JQL overview**
https://www.atlassian.com/software/jira/guides/jql/overview

- **Jira permission types**
https://support.atlassian.com/jira-cloud-administration/docs/types-of-permissions-in-jira/

- **Jira automation getting started (Data Center docs; verify version)**
https://confluence.atlassian.com/automation0800/get-started-with-jira-automation-1189780056.html

- **Atlassian Learning catalogue**
https://learning.atlassian.com/learning/catalog?product=Jira

Final advice

Learn Jira by running a small realistic workflow end to end. Create a work item, move it through a workflow, search for it with JQL, show it on a dashboard, and inspect the history. Then learn configuration and automation in a test project. Always distinguish product features from your organisation’s conventions, and verify high-impact changes against current official documentation.

> Review note: this handbook was structured as a practical guide and checked for internal consistency, terminology distinctions, example-query readability, and inclusion of diagrams, workflows, practical scenarios, interview questions, and official resources. Jira changes over time and customer configurations differ; URLs and exact UI paths should be rechecked before using this as a formal internal standard.